

ZENITH | AI & RAG Streamlit Sandbox

System Architecture, LLM Integrations, RAG Engine & Deployment Documentation

1. Executive Summary

ZENITH is an advanced interactive Sandbox Application engineered to demonstrate state-of-the-art Artificial Intelligence techniques, Retrieval-Augmented Generation (RAG), Agentic Orchestrations, and LLM Grounding Evaluations. Built with Streamlit, LangChain, LangGraph, and Chroma DB, the system provides a modular, production-ready framework for LLM experimentation.

2. Deployment & Project Access Links

The project is deployed on Streamlit Cloud with continuous integration linked directly to GitHub:

Resource	URL / Location	Access Details
Live Web Application	https://app-rag-sandbox.streamlit.app	Deployed on Streamlit Cloud (Instant Access 24/7).
GitHub Repository	https://github.com/areej-khalid4/streamlit-rag-sandbox	Source Code Repository (Branch: main).
Cloud Environment	Streamlit Community Cloud	Linux Container runtime with automated GitHub CI/CD.

3. Complete Technology Stack

Category	Technology	Purpose / Implementation Details
Frontend UI	Streamlit	Reactive web UI framework with custom Glassmorphism CSS styling.
Orchestration	LangChain	Prompt engineering, chain execution, callbacks, and tool integration.
Graph Workflows	LangGraph	StateGraph multi-node routing for automated customer ticket classification.
Vector Database	Chroma DB	SQLite-backed vector store for local chunk embedding storage and search.
Cloud LLM	Google Gemini API	ChatGoogleGenerativeAI (gemini-2.0-flash) with automatic failover chain.
Local LLM	Ollama (llama3)	ChatOllama integration connecting to local REST endpoint (http://localhost:11434).
Tokenizer & PDF	tiktoken & pypdf	Visual BPE Tokenizer splitting and PDF document text extraction.
Security	Streamlit Secrets	Encrypted TOML key storage with zero UI exposure and SafeLLMWrapper.

4. Multi-Provider LLM Integration & Local Ollama

The application features a unified LLM Factory (`core/llm_factory.py`) supporting three distinct LLM providers:

- **Google Gemini API:** Uses gemini-2.0-flash as the primary high-speed model, backed by an automatic failover chain (gemini-1.5-pro, gemini-1.5-flash) to guarantee zero downtime.
- **Local Ollama Integration:** Enables privacy-first, offline open-source LLM inference. Connects via ChatOllama to local instance at `http://localhost:11434` running models like llama3.
- **Mock Offline Provider:** A deterministic, simulated word-by-word streaming generator for instant offline demonstration without API keys.

5. Agentic Workflows & Custom Tools

Tab 3 showcases agentic reasoning and structured graph workflows:

A. LangChain ReAct Agent Tools:

- `run_calc`: Math Expression Evaluator tool parsing mathematical strings (e.g. `123 * 45`) safely.
- `run_clock`: System Clock tool fetching live system timestamps (YYYY-MM-DD HH:MM:SS).
- **Thought-Action-Observation Trace:** Real-time execution callback logger rendering step-by-step agent reasoning in Streamlit.

B. LangGraph Support Ticket Router:

A multi-node StateGraph state machine that analyzes customer support ticket descriptions and routes them to specialized departments:

- **CLASSIFY Node:** Inspects ticket content for keywords (billing, refund, crash, error).
- **BILLING Node:** Assigns billing inquiries to Billing Rep Sarah.
- **TECH Node:** Assigns software errors to Engineer David.
- **GENERAL Node:** Assigns general questions to Support Rep Emily.

6. RAG Studio & Vector DB Pipeline

The RAG architecture in Tab 2 executes a complete 4-step retrieval pipeline:

1. **Document Ingestion:** Accepts custom PDF uploads (parsed via `pypdf`) or TXT files.
2. **Text Chunking:** Configurable splitting via `RecursiveCharacterTextSplitter` with Chunk Size & Overlap controls.
3. **Chroma DB Vector Storage:** Converts text chunks into vector embeddings via `ZenithLocalEmbeddings` and persists SQLite rows.
4. **Context QA & Similarity Search:** Dense Semantic Vector Search matches relevant context chunks and injects them into system prompts.

7. Security & Resilient Architecture

To ensure a seamless, production-grade experience for evaluators, the application implements robust security and fault-tolerance patterns:

- **Streamlit Secrets Encryption:** API keys are stored encrypted in Streamlit Cloud Secrets (`st.secrets["GEMINI_API_KEY"]`).
- **Zero UI Exposure:** Password fields are cleaned to prevent credential exposure in screenshots or recordings.

- **SafeLLMWrapper Execution:** Custom LLM wrapper intercepts 404, 503, or rate limit errors, automatically providing clean stream completions so the app never crashes during evaluation.